

Project Report

Script-Controlled ACL – Restrict Record Access Based on Field Value

ServiceNow Access Control Implementation

Project Information	Details
Project Title	Script-Controlled ACL – Restrict Record Access Based on Field Value
Platform	ServiceNow
Project Type	Access-control / Service Management Security
Team ID	SWTID-2026-7951
Team Members	Jenifer.N, Sowmiya.A, Sujitha Blessy.P,Yazhini.N ,Priya Dharshini.P
Guide / Faculty	
Academic Year	2026

1. INTRODUCTION

1.1 Project Overview

This project implements a script-controlled Access Control List (ACL) in ServiceNow to enforce record-level visibility based on the value of a Branch field. The core scenario is that users belonging to the EEE branch are permitted to view records whose Branch value is EEE, while administrators retain full access. The ACL makes the authorization decision at record level instead of relying only on broad table permissions.

1.2 Purpose

The purpose is to demonstrate how conditional authorization can protect records from unauthorized viewing. The solution aims to reduce accidental data exposure, provide predictable branch-based access, preserve administrator access, and provide a maintainable pattern that can be extended to other branch-specific security rules.

2. IDEATION PHASE

2.1 Problem Statement

Users may be able to view records outside their authorized branch when access is controlled only through general permissions. In the proposed scenario, the system must restrict normal users so that only users belonging to the EEE branch can view records where Branch = EEE. Administrators must retain full access for legitimate administrative activities.

2.2 Empathy Map Canvas

Area	Project-specific observation
Says	Users expect the system to show only records relevant to their branch and administrators expect uninterrupted management access.
Thinks	Users may be concerned about whether confidential branch records are protected; administrators need to know that the restriction will not block their work.
Does	Users log in, search for records and open records. Administrators access and manage records across branches.
Feels	Authorized users should feel confident that access is controlled; unauthorized users should receive a clear access restriction.
Pain Point	Broad permissions can expose records beyond the user's branch or

	require manual checking.
Need	Automatic, record-level and consistent authorization based on the Branch field.

2.3 Brainstorming

- Use a scripted ACL to evaluate the logged-in user and the requested record.
- Compare the user's branch with the record's Branch value.
- Provide an explicit administrator exception.
- Test positive, negative and edge-case access scenarios.
- Keep the logic simple so it can be extended to additional branches.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	User Action	System Behaviour	Expected Experience
Login	User signs in	ServiceNow establishes user context	User identity is recognized.
Find Record	User searches or opens a record	Record is requested	User can attempt access normally.
Authorization	ACL evaluates request	User role/branch and record Branch are checked	Decision is automatic.
Access	Condition passes or fails	Record is displayed or access is denied	Only authorized records are visible.
Administration	Administrator opens records	Administrator exception is evaluated	Administrator retains full access.

3.2 Solution Requirement

ID	Requirement	Type
FR-01	Identify the logged-in user and authorization context.	Functional
FR-02	Read the Branch value of the requested record.	Functional
FR-03	Allow an EEE user to view a record when Branch = EEE.	Functional
FR-04	Deny a normal user who does not satisfy the required branch condition.	Functional
FR-05	Allow administrators to view records across branches.	Functional
FR-06	Apply the decision automatically when a record is requested.	Functional
NFR-01	Access decisions should be consistent for the same user and record conditions.	Non-functional
NFR-02	ACL logic should avoid unnecessary processing and queries.	Non-functional
NFR-03	Authorization rules should be maintainable and extensible.	Non-functional
NFR-04	Unauthorized users must not receive protected record data.	Non-functional

3.3 Data Flow Diagram

Level 0 flow: User / Administrator → ServiceNow Record Request → Script-Controlled ACL → User/Role Context + Record Branch → Allow or Deny → Record View.

[Insert the DFD image created in the Requirement Analysis phase here.]

3.4 Technology Stack

Layer	Technology / Component	Purpose
Application Platform	ServiceNow	Hosts the records, users, roles and access-control configuration.
Security	Access Control List (ACL)	Enforces record-level authorization.
Logic	ServiceNow server-side scripting	Evaluates role/branch and record field conditions.
Identity	ServiceNow user and role records	Provides logged-in user and role context.
Data	ServiceNow application table	Stores records including the Branch field.
UI	ServiceNow web interface	Used to access lists and individual records.

External API	Not required	The described solution can be implemented without an external API.
ML	Not applicable	The project is rule/script-based, not a machine-learning system.

4. PROJECT DESIGN

4.1 Problem Solution Fit

Canvas Area	Project Analysis
Customer Segment	EEE branch users and administrators.
Customer Constraints	Users must not see records outside their authorization; administrators require broader access.
Available Solutions	Broad role permissions, manual checking, application filtering or ACL rules.
Root Cause	General access permissions do not necessarily encode the required record Branch condition.
Behaviour	Users request records and expect the platform to enforce access automatically.
Triggers	Opening a branch-specific record or an administrator performing management.
Solution	A scripted ACL evaluates user context against the record's Branch field.
Outcome	EEE users can access EEE records; unauthorized users are denied; administrators retain full access.

4.2 Proposed Solution

The proposed solution is a ServiceNow scripted ACL on the relevant record access operation. The script checks the current user's authorization context and the Branch value of the target record. The decision follows the defined policy: administrator → allow; otherwise, EEE user with Branch = EEE → allow; otherwise → deny. The logic should be implemented only after confirming the actual user/branch field names and role configuration in the target ServiceNow instance.

4.3 Solution Architecture

Architecture components:

- User / Administrator: initiates a record-view request.
- ServiceNow UI: presents lists and records.
- ACL Engine: invokes the access rule when the protected record is requested.
- User / Role Context: supplies the user's identity, role and branch information.
- Target Record: supplies the Branch field value.
- Authorization Decision: returns Allow or Deny before the protected record is exposed.

[Insert the Solution Architecture diagram created in the Project Design phase here.]

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Sprint	Focus	Story Points	Planned Output
Sprint 1	User/role context and Branch validation	8	Access context and record-field checks prepared.
Sprint 2	Core scripted ACL behaviour	8	EEE allow, unauthorized deny and administrator allow behaviour implemented.
Sprint 3	Testing, documentation and release	5	Functional/performance validation, UAT and final documentation.

Total planned backlog: 21 story points. Team member names and actual completion dates should be entered from the project planning sheet.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Test ID	Scenario	Expected Result	Actual Result
FT-01	EEE user → EEE record	Allow	To be filled
FT-02	Non-EEE user → EEE record	Deny	To be filled
FT-03	Administrator → EEE record	Allow	To be filled
FT-04	Administrator → other branch record	Allow	To be filled
FT-05	EEE user → non-EEE record	No unauthorized exposure	To be filled
FT-06	Empty/unexpected Branch	No unintended access for normal user	To be filled
PT-01	Representative record opening	No unnecessary noticeable delay	To be measured
PT-02	Repeated/multiple-user access	Consistent results without abnormal slowdown	To be measured

7. RESULTS

7.1 Output Screenshots

Insert the following screenshots from the actual ServiceNow instance:

- ACL configuration showing the table, operation, condition/script and active state.
- EEE user viewing an EEE record successfully.
- Unauthorized/non-EEE user being denied access to the protected record.
- Administrator viewing an EEE record successfully.
- Administrator viewing a non-EEE record successfully.
- Test result or validation evidence.

8. ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages / Limitations
Provides record-level access control.	Requires correct user/branch and role configuration.
Reduces unauthorized record visibility.	A poorly written script can create unintended access or denial.
Preserves administrator access.	Testing must cover positive, negative and edge cases.
Can be extended to other branch rules.	The exact field and role names are instance-specific.
Centralizes the decision in the ACL layer.	External systems are not covered unless they use the same authorization path.

9. CONCLUSION

The project demonstrates how a script-controlled ServiceNow ACL can enforce conditional record visibility. By evaluating the user's authorization context together with the Branch field of the requested record, the solution provides a clear record-level access rule. The administrator exception ensures that legitimate administrative access is preserved. Functional testing and UAT can verify that authorized users are allowed, unauthorized users are denied, and the behaviour remains consistent.

10. FUTURE SCOPE

- Extend the rule from EEE to multiple branches using configurable branch mappings.
- Support multiple roles and more granular authorization conditions.
- Create reusable security logic for additional sensitive tables.
- Add automated regression tests for ACL scenarios.
- Introduce centralized audit/reporting of authorization decisions where supported by the ServiceNow implementation.
- Add a configuration-driven approach so branch rules can be changed without rewriting core logic.

11. APPENDIX

11.1 Source Code (if any)

Insert the final ServiceNow ACL script here after implementation. Do not include passwords, tokens or other credentials.

11.2 Dataset Link

Not applicable. The project uses ServiceNow records and user/role configuration rather than an external dataset.

11.3 GitHub & Project Demo Link

GitHub: _____

Project Demo / ServiceNow Instance: _____